

Race conditions and the critical section problem, Deadlock conditions (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait) , Resource starvation and the aging technique solution ., Priority inversion and basic concurrency bugs

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

int counter = 0;

/* Function executed by both threads */
void *increase(void *arg) {

    for (int i = 0; i < 100000; i++) {
        counter++;
    }

    return NULL;
}

int main() {

    pthread_t t1, t2;

    printf("CONCURRENCY DEMONSTRATION\n\n");

    /* ----- RACE CONDITION ----- */
    printf("1. RACE CONDITION\n");
```

```

counter = 0;

pthread_create(&t1, NULL, increase, NULL);
pthread_create(&t2, NULL, increase, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Expected value = 2000000\n");
printf("Actual value   = %d\n", counter);

printf("\nBoth threads access the same variable.\n");
printf("This can cause a race condition.\n");

/* ----- CRITICAL SECTION ----- */

printf("\n2. CRITICAL SECTION\n");

printf("The statement counter++ is the critical section.\n");
printf("Only one thread should access shared data at a time.\n");

/* ----- DEADLOCK ----- */

```

```

printf("\n3. DEADLOCK CONDITIONS\n");

printf("1. Mutual Exclusion\n");
printf("2. Hold and Wait\n");
printf("3. No Preemption\n");
printf("4. Circular Wait\n");

printf("\nIf all four conditions occur together, deadlock can occur.\n");

/* ----- STARVATION ----- */

printf("\n4. STARVATION\n");

printf("A low-priority process may wait for a very long time.\n");

printf("AGING SOLUTION:\n");
printf("Increase the priority of a process as it waits.\n");

printf("Example:\n");
printf("Initial priority = 5\n");
printf("After waiting    = 4\n");
printf("After more wait  = 3\n");

```

```
/* ----- PRIORITY INVERSION ----- */  
  
printf("\n5. PRIORITY INVERSION\n");  
  
printf("Low-priority process holds a resource.\n");  
printf("High-priority process needs the same resource.\n");  
printf("Medium-priority process keeps running.\n");  
  
printf("\nPriority inheritance can solve this problem.\n");  
  
printf("\nConcurrency demonstration completed.\n");  
  
return 0;  
}
```

Output:

CONCURRENCY DEMONSTRATION

1. RACE CONDITION

Expected value = 200000

Actual value = 116403

Both threads access the same variable.

This can cause a race condition.

2. CRITICAL SECTION

The statement `counter++` is the critical section.

Only one thread should access shared data at a time.

3. DEADLOCK CONDITIONS

1. Mutual Exclusion

2. Hold and Wait

3. No Preemption

4. Circular Wait

If all four conditions occur together, deadlock can occur.

4. STARVATION

A low-priority process may wait for a very long time.

AGING SOLUTION:

Increase the priority of a process as it waits.

Example:

Initial priority = 5

After waiting = 4

After more wait = 3

5. PRIORITY INVERSION

Low-priority process holds a resource.

High-priority process needs the same resource.

Medium-priority process keeps running.

Priority inheritance can solve this problem.

Concurrency demonstration completed.

geya@LenovoL480:~/OS/Practical/Week 7\$ |